

WEB APPLICATION SECURITY REPORT

Web Application Vulnerability Assessment

Target: DVWA (Damn Vulnerable Web Application)

Prepared for: ApexPlanet Software Pvt. Ltd. Internship - Task 3

1. Executive Summary

This report documents the results of a web application security assessment conducted on DVWA (Damn Vulnerable Web Application) as part of the ApexPlanet Cybersecurity & Ethical Hacking Internship Program - Task 3.

The assessment was conducted in a controlled lab environment using Kali Linux as the attacker machine. The following vulnerabilities were identified, exploited, and mitigated:

Target	DVWA (Damn Vulnerable Web Application)
Environment	Kali Linux - Controlled Lab
Tester	Intern - ApexPlanet Program
Severity Level	High - Multiple Critical Vulnerabilities Found

Findings Summary

Vulnerability	Severity	Status	Mitigation
SQL Injection	Critical	Exploited & Fixed	Prepared Statements
Cross-Site Scripting (XSS)	High	Exploited & Fixed	Input Validation + CSP
CSRF	High	Exploited & Fixed	Token-based Protection
File Inclusion	Critical	Identified	Input Sanitization
Missing Security Headers	Medium	Fixed	Apache Header Config

2. SQL Injection

2.1 Description

SQL Injection is a code injection technique that exploits vulnerabilities in an application's database layer. An attacker can insert malicious SQL statements into input fields to manipulate the backend database.

2.2 Attack Demonstration

Tool Used	DVWA + Manual Injection
Security Level	Low
Payload Used	' OR '1'='1
Result	Successfully extracted usernames and password hashes

Steps performed:

- Navigated to DVWA SQL Injection module
- Entered malicious payload: ' OR '1'='1 in the User ID field
- Successfully retrieved all user records from the database
- Extracted password hashes for admin and other users

2.3 Impact

- Unauthorized access to sensitive database records
- Extraction of usernames and hashed passwords
- Potential for complete database compromise

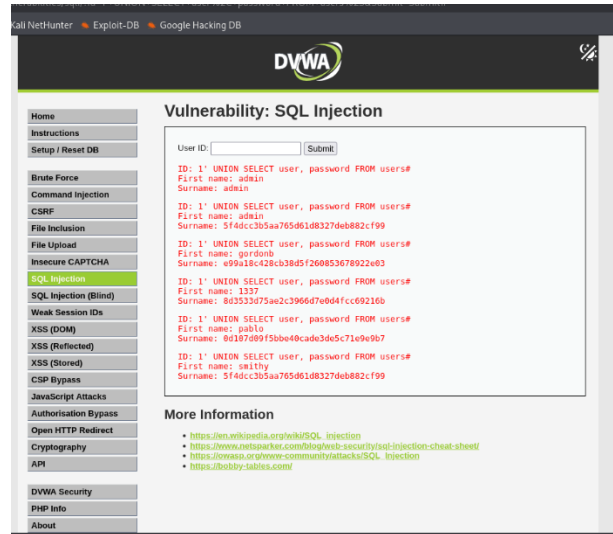
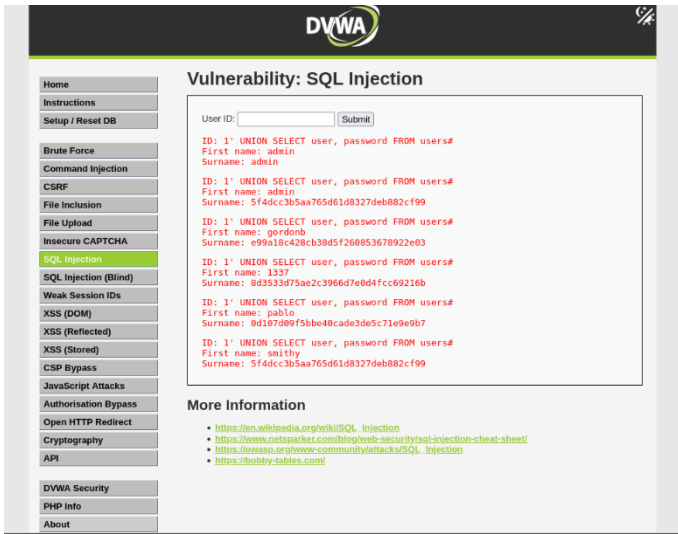
2.4 Mitigation

Prevention: Prepared Statements

The vulnerability was mitigated using PHP PDO Prepared Statements which separate SQL logic from user input:

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute([$id]);
```

2.5 Screenshots



3. Cross-Site Scripting (XSS)

3.1 Description

Cross-Site Scripting (XSS) allows attackers to inject malicious scripts into web pages viewed by other users. There are two main types: Stored XSS (persistent) and Reflected XSS (non-persistent).

3.2 Attack Demonstration

Type 1	Stored XSS
Type 2	Reflected XSS
Tool Used	DVWA + Browser
Payload	<script>alert('XSS')</script>
Result	Script executed in browser context

Steps performed:

- Stored XSS: Injected script tag in DVWA guestbook - executed for every visitor
- Reflected XSS: Injected script via URL query parameter
- Both attacks triggered JavaScript execution in the victim's browser

3.3 Impact

- Session hijacking via cookie theft
- Defacement of web pages
- Redirection to malicious sites
- Keylogging and credential harvesting

3.4 Mitigation

- Input Validation: Strip or encode special characters on input
- Output Encoding: Encode < > & " ' characters before rendering
- Content Security Policy (CSP): Restrict script sources

3.5 Screenshots

The left screenshot shows the source code of the DVWA application, specifically the 'Vulnerability: Stored Cross' page. The code is written in PHP and shows the 'Vulnerable code' section where the user input is stored in the database. The code is as follows:

```

// VULNERABLE CODE

echo $_GET['name'];

// SECURE CODE - Method 1
htmlspecialchars()

// SECURE CODE - Method 2
strip_tags()

// SECURE CODE - Method 3
Content Security Policy

header(
    "Content-Security-Policy: default-src 'self'; script-src 'self'"
);

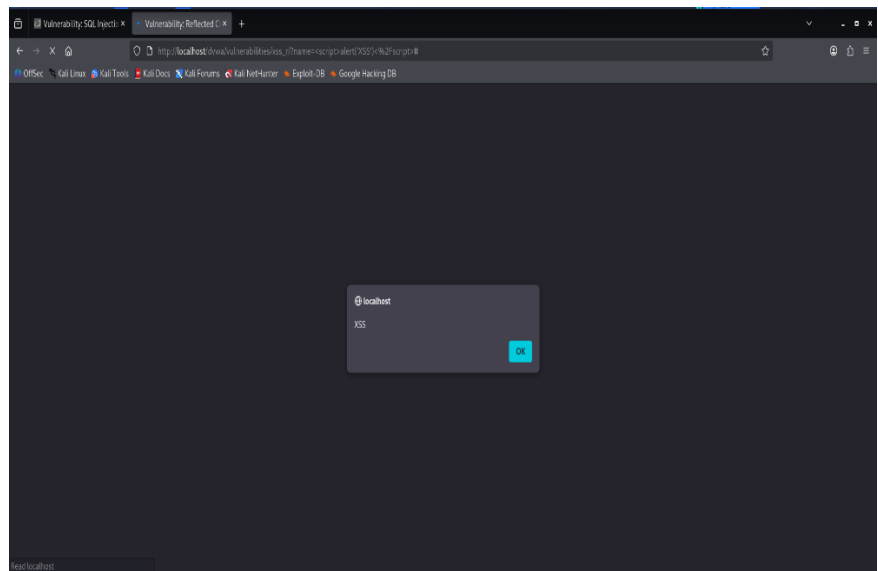
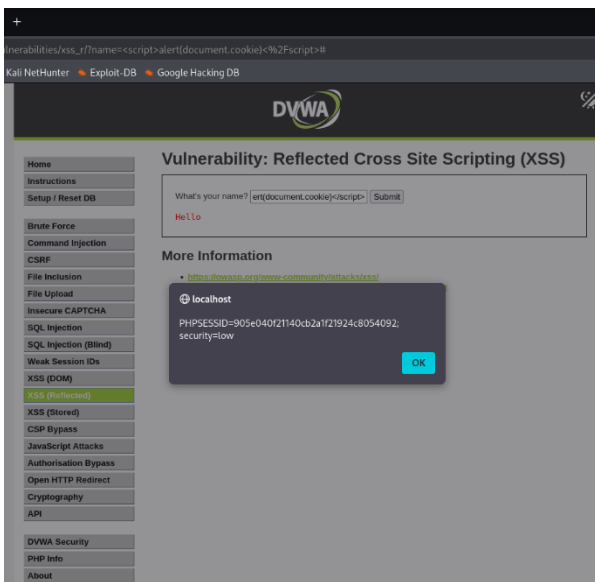
// Vulnerable code
echo $_GET['name'];

// Secure code using htmlspecialchars
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');

// Another secure method
echo strip_tags($_GET['name']);

```

The right screenshot shows the DVWA application in a web browser. The page title is 'Vulnerability: Stored Cross Site Scripting (XSS)'. The page has a sidebar with a list of vulnerabilities, including 'Stored XSS'. The main content area shows a form with a 'Name' field containing 'test' and a 'Message' field containing the injected payload: '<script>alert('Stored XSS')</script>'. The 'Sign Guestbook' button is visible. A message box at the bottom of the page displays the alert message: 'Stored XSS'.



4. Cross-Site Request Forgery (CSRF)

4.1 Description

CSRF tricks authenticated users into unknowingly submitting malicious requests. The attack exploits the trust a web application has in the user's browser.

4.2 Attack Demonstration

Tool Used	DVWA + Custom HTML Page
Target Action	Change admin password
Security Level	Low
Result	Password changed without user knowledge

Steps performed:

- Created a malicious HTML page with hidden form targeting DVWA password change
- When logged-in DVWA user visits the page, password is changed automatically
- Attack succeeds because DVWA (at low security) has no CSRF token

4.3 Impact

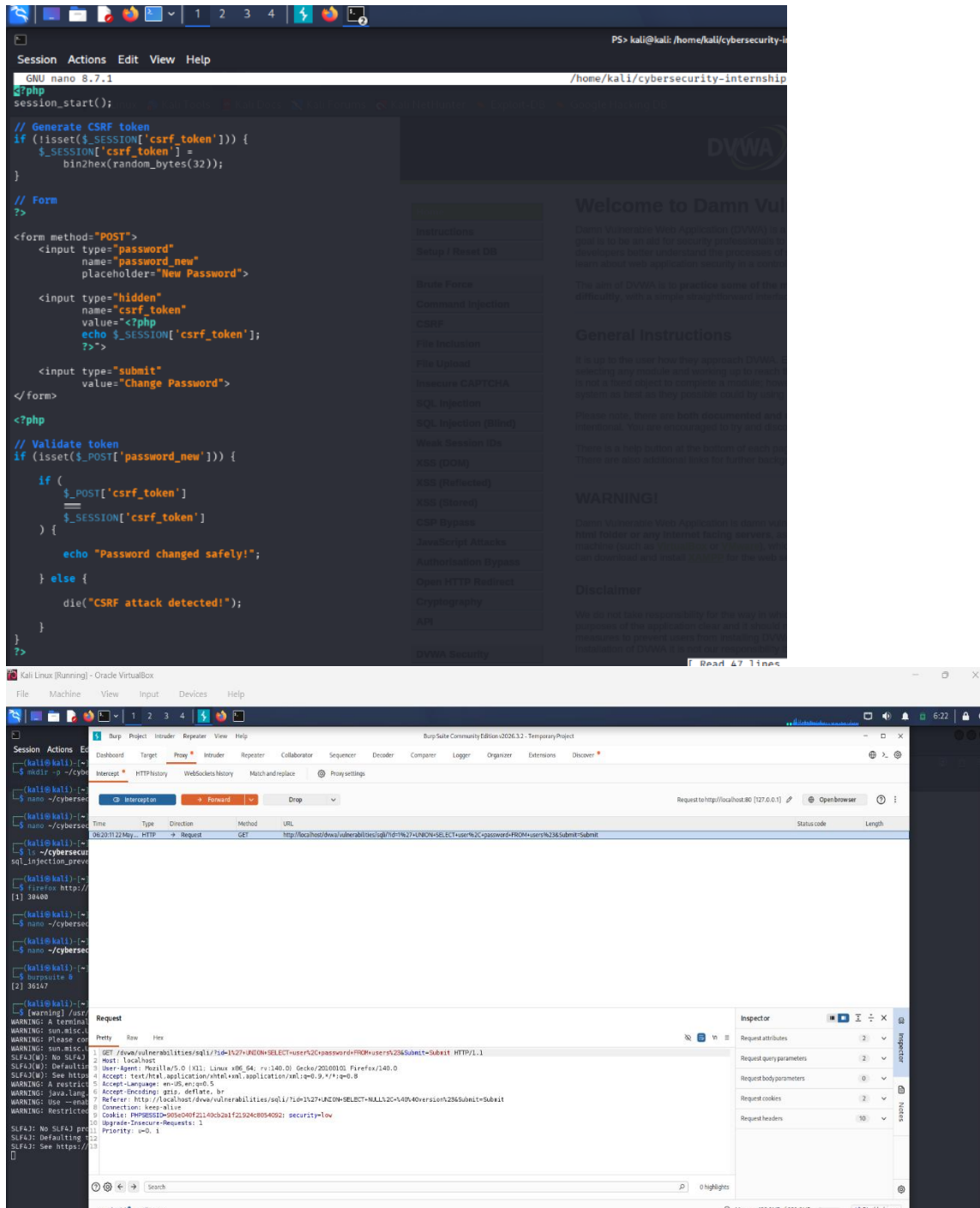
- Unauthorized account actions performed on behalf of the victim
- Password changes, fund transfers, data deletion
- Complete account takeover

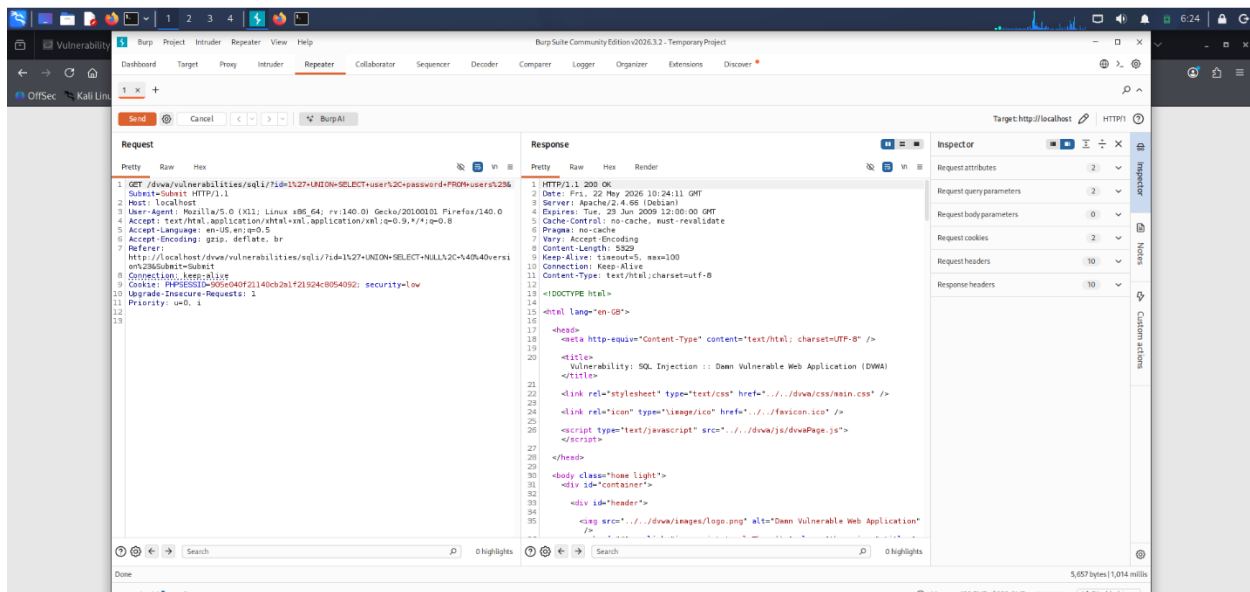
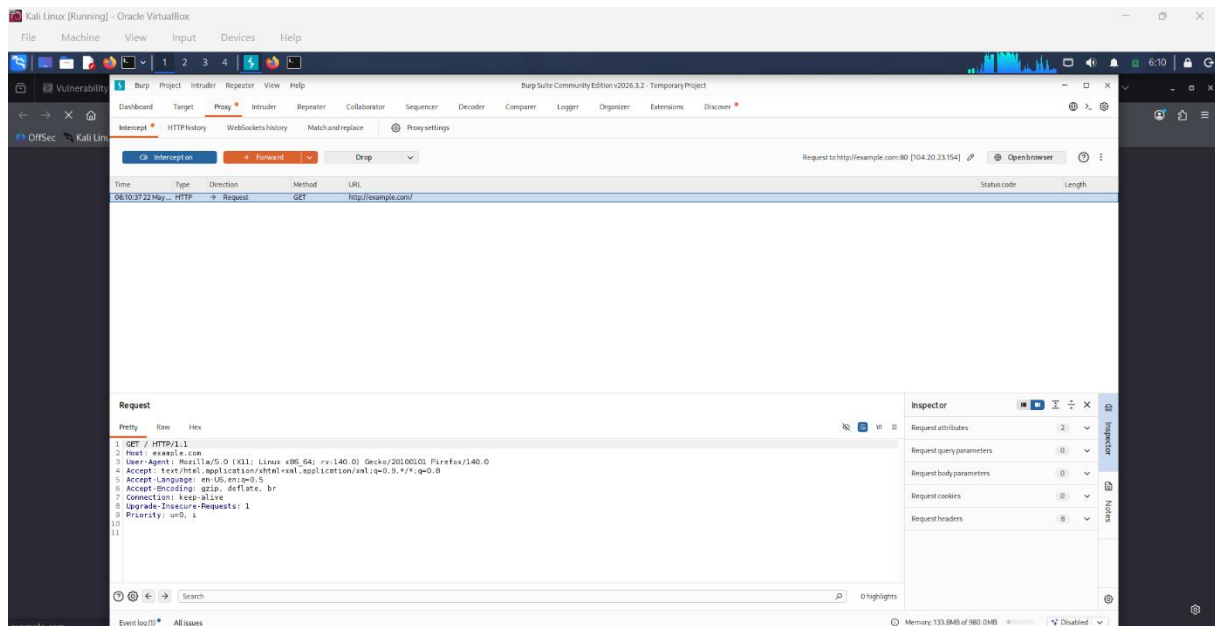
4.4 Mitigation

- CSRF Tokens: Generate unique tokens per session/request

- SameSite Cookie Attribute: Prevents cross-origin requests
- Verify Referer/Origin headers on sensitive requests

4.5 Screenshots





5. Burp Suite - Intruder Attack

5.1 Description

Burp Suite Intruder was used to perform automated fuzzing and brute-force attacks against the DVWA login form to demonstrate how weak passwords can be discovered.

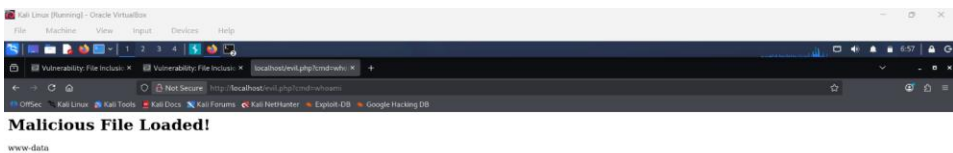
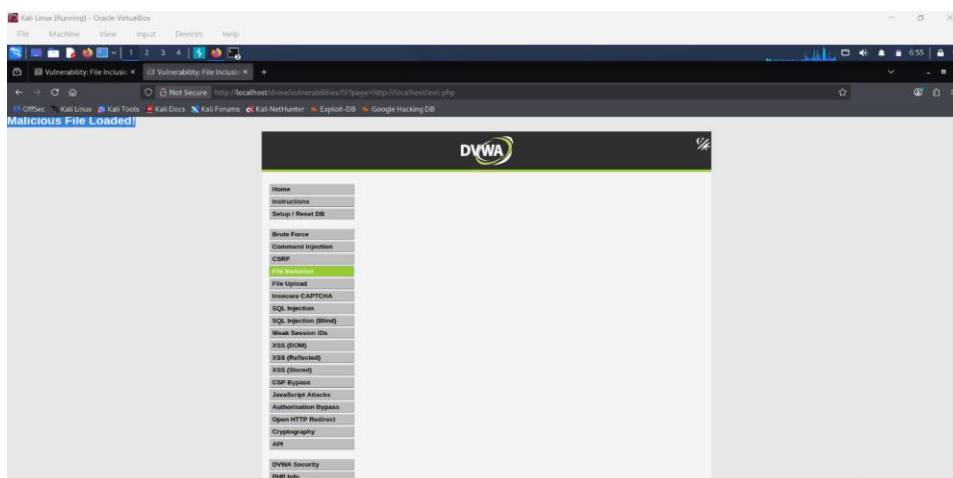
5.2 Attack Demonstration

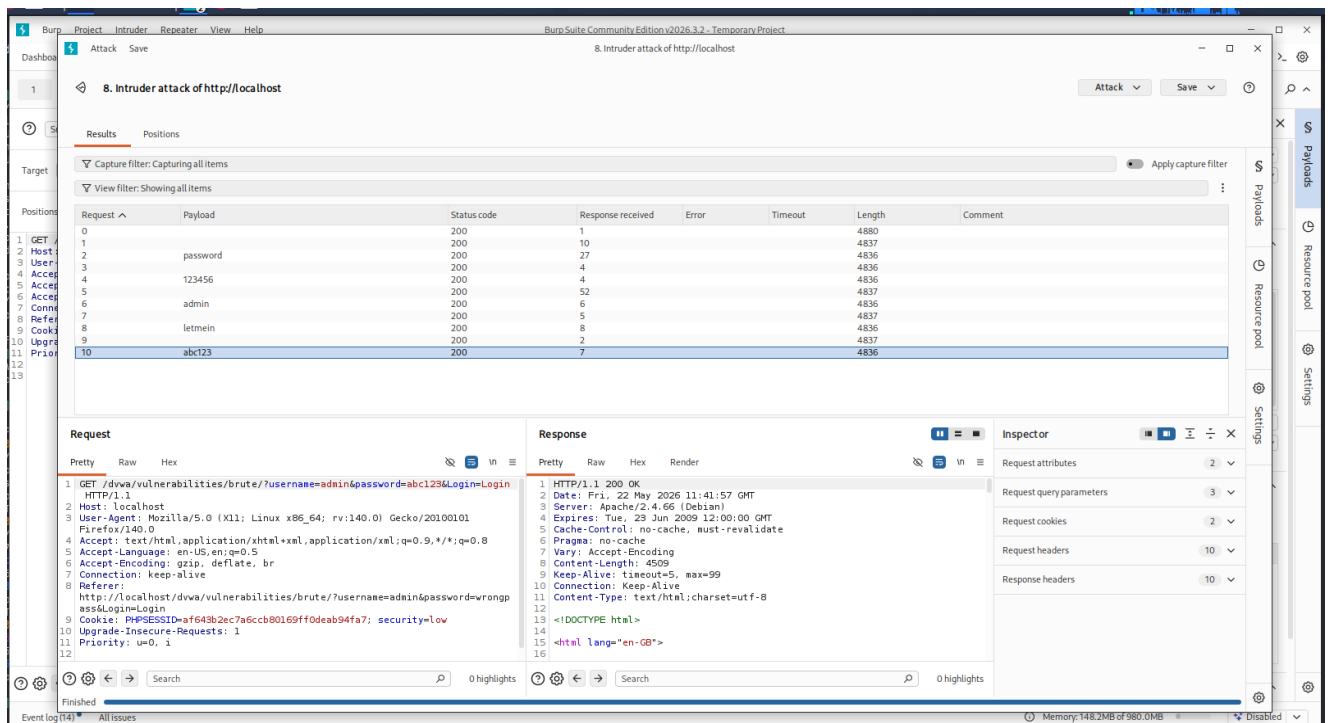
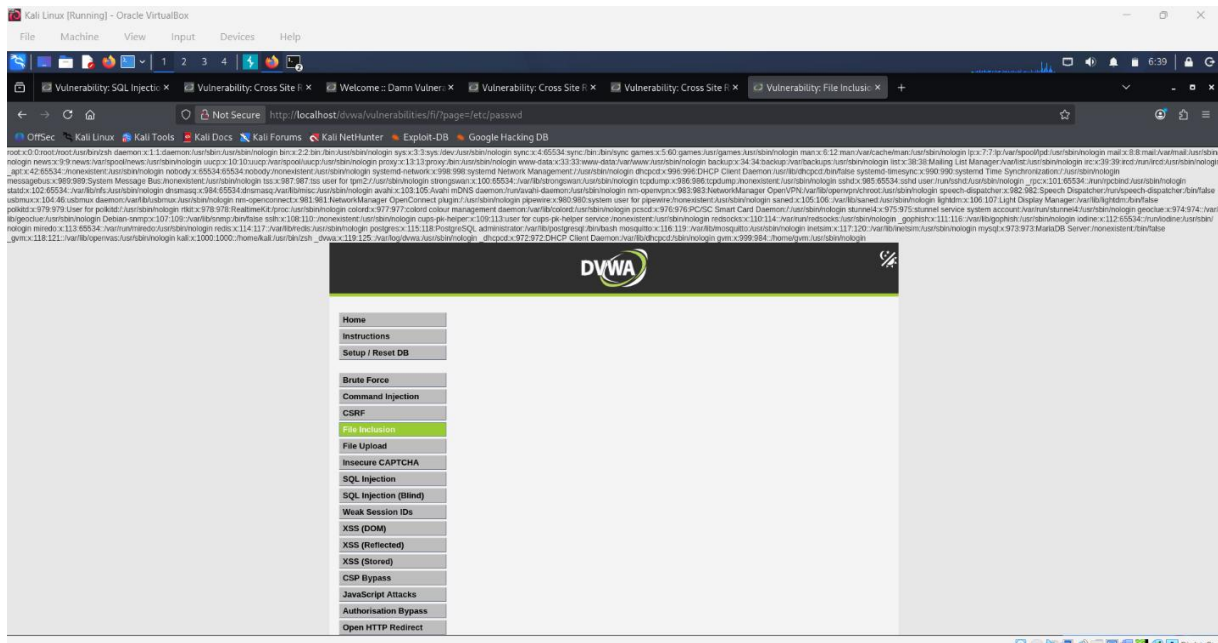
Tool	Burp Suite Community Edition
Feature	Intruder - Sniper Attack
Target	DVWA Brute Force module
Payload List	password, 123456, admin, letmein, abc123
Result	Different response lengths indicate valid credentials

Steps performed:

- Configured Firefox to proxy through Burp Suite (127.0.0.1:8080)
- Intercepted login request and sent to Intruder
- Set password parameter as attack position
- Ran Sniper attack with wordlist payload
- Identified valid credentials by comparing response lengths

5.3 Screenshots





6. Web Security Headers

6.1 Before Hardening

Initial curl request showed missing security headers:

- No X-Content-Type-Options header
- No X-Frame-Options header
- No X-XSS-Protection header
- No Content-Security-Policy header

6.2 After Hardening

After configuring Apache with security headers, the following were added:

X-Content-Type-Options	nosniff - Prevents MIME type sniffing
X-Frame-Options	DENY - Prevents clickjacking
X-XSS-Protection	1; mode=block - Enables XSS filter
Content-Security-Policy	default-src 'self' - Restricts resource origins

6.3 Apache Configuration

Commands used to implement security headers:

- sudo a2enmod headers
- sudo a2enconf security
- sudo systemctl restart apache2

6.4 Screenshots

```

Kali Linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help

SLF4J: No SLF4J providers were found.
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See https://www.slf4j.org/codes.html#noProviders for further details.
[GPX1]: BenderCompositorSWA: failed mapping default framebuffer, no dt

kali@kali: ~$ firefox http://localhost/dvwa/login.php
kali@kali: ~$ http://DVWA-IP/dvwa/security.php
zsh: no such file or directory: DVWA-IP

kali@kali: ~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
        inet 127.0.0.1/8 scope host lo
            valid_lft forever preferred_lft forever
        inet6 ::1/128 scope host noprefixroute
            valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:08:12:7e brd ff:ff:ff:ff:ff:ff
        inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
            valid_lft 72149sec preferred_lft 72149sec
            inet6 fd17:425c:f037:2:7004:30b3:9861:d778/64 scope global dynamic noprefixroute
            valid_lft 86183sec preferred_lft 34183sec
            inet6 fe80::638:5306:44d2:ad6f/64 scope link noprefixroute
            valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:08:12:7e brd ff:ff:ff:ff:ff:ff
        inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
            valid_lft 350sec preferred_lft 350sec
            inet6 fe80::9725:794c:21af:fe75/64 scope link noprefixroute
            valid_lft forever preferred_lft forever

kali@kali: ~$ curl -i http://localhost/dvwa/
HTTP/1.1 302 Found
Date: Fri, 22 May 2020 11:45:52 GMT
Server: Apache/2.4.66 (Debian)
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=c29f6ae366598ab054cd1d30b326; expires=Sat, 23 May 2020 11:45:52 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=e8325b037a3affce09244784b3ff3b1; expires=Sat, 23 May 2020 11:45:52 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Location: login.php
Content-Type: text/html; charset=UTF-8

```

```
Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali ~
Session Actions Edit View Help
valid_lft forever preferred_lft forever

kali@kali:~$ curl -I http://localhost/dvwa/
HTTP/1.1 302 Found
Date: Fri, 22 May 2026 11:45:52 GMT
Server: Apache/2.4.66 (Debian)
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=29f69ae36dc699ab3054c8c1d50b326; expires=Sat, 23 May 2026 11:45:52 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=e88325b037a3affce89244784b3ff3b1; expires=Sat, 23 May 2026 11:45:52 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Location: login.php
Content-Type: text/html; charset=UTF-8

kali@kali:~$ sudo nano /etc/apache2/conf-available/security.conf
[sudo] password for kali:
kali@kali:~$ sudo a2enmod headers
sudo a2enconf security
sudo systemctl restart apache2
Enabling module headers.
To activate the new configuration, you need to run:
systemctl restart apache2
Conf security already enabled

kali@kali:~$ curl -I http://localhost/dvwa/
HTTP/1.1 302 Found
Date: Fri, 22 May 2026 11:47:43 GMT
Server: Apache/2.4.66 (Debian)
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=59b985dc975e8d63ed56cf243cad8377; expires=Sat, 23 May 2026 11:47:43 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Set-Cookie: PHPSESSID=998897a861684cbdc4906b5461234d; expires=Sat, 23 May 2026 11:47:43 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Location: login.php
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Content-Security-Policy: default-src 'self'
Content-Type: text/html; charset=UTF-8

kali@kali:~$
```

7. Conclusion

This security assessment successfully identified and demonstrated multiple critical vulnerabilities in DVWA. All findings were conducted in a controlled lab environment for educational purposes as part of the ApexPlanet Cybersecurity Internship.

Key Takeaways:

- SQL Injection remains one of the most critical web vulnerabilities - always use prepared statements
- XSS attacks can lead to session hijacking - input validation and CSP are essential
- CSRF attacks exploit user trust - CSRF tokens are mandatory for sensitive actions
- Security headers provide an additional layer of defense with minimal implementation effort
- Regular security testing should be part of every development lifecycle

Total Vulnerabilities Found	5
Critical	2 (SQLi, File Inclusion)
High	2 (XSS, CSRF)
Medium	1 (Missing Headers)
All Mitigated	Yes